

Advanced Compilers

Project 1

Virtual Machine & Optimization Laboratory
Dept. of Electrical and Computer Engineering
Seoul National University



Index

1. **Introduction to LLVM**
2. **Environment setup**
3. **Project 1 : Printing CFG**
4. **Appendix**
 - Build from source
 - LLVM details



Preface

INTRODUCTION TO LLVM

What is LLVM?

LLVM

- Compiler middle- and back-end infrastructure

LLVM IR

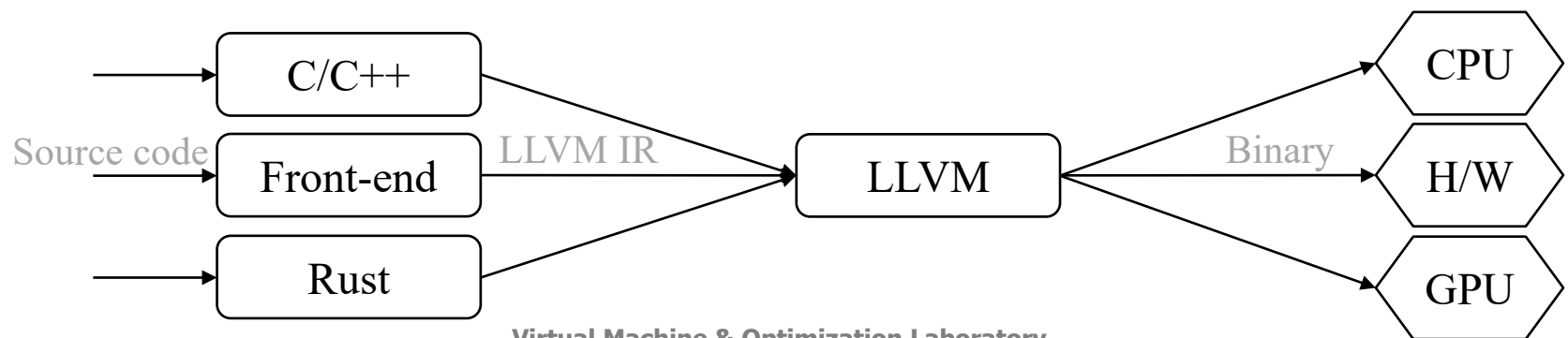
- Static single-assignment (SSA) intermediate representation (IR)

Middle-end

- Do machine-**independent** optimizations

Back-end

- Do machine-**dependent** optimizations
- Machine **code generation**



LLVM Pipeline

Front-end

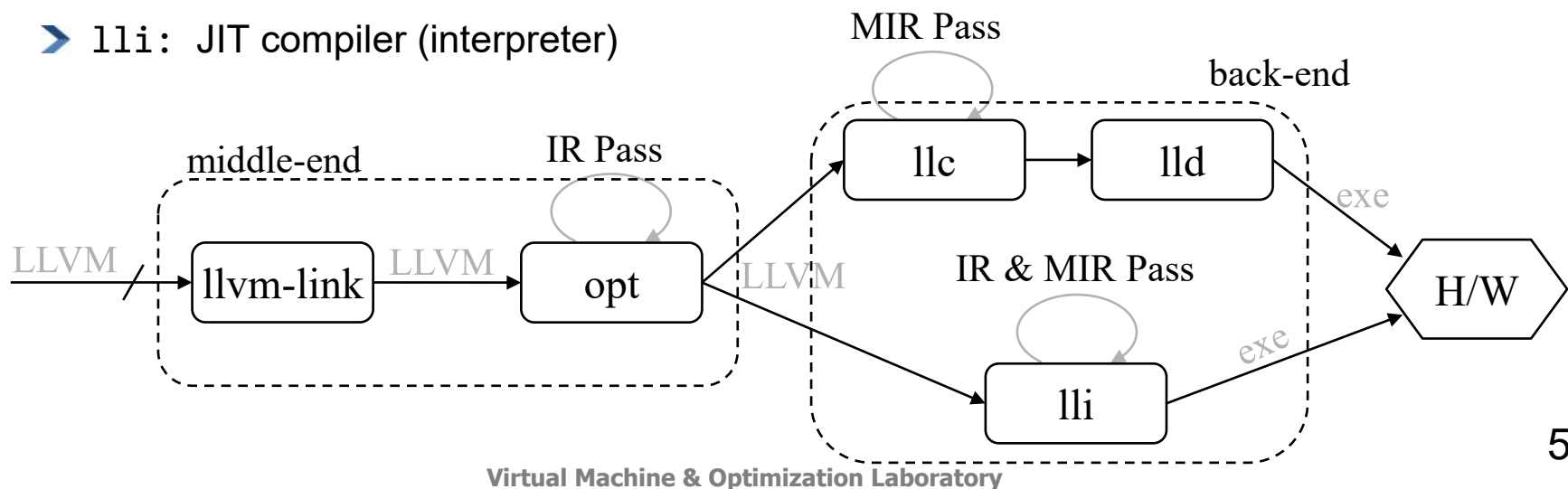
- Convert source code into LLVM IR using code structures (not an LLVM spec)

Middle-end

- `llvm-link`: IR-level linker
- `opt`: IR optimizer

Back-end

- `llc`: IR compiler
- `lld`: Linker
- `lli`: JIT compiler (interpreter)



LLVM IR

Properties

- Unified
 - IR is used for program's whole life-cycle (middle-end, back-end, profiling, ...)
- 3-address form
 - e.g., $x = OP\ a,\ b$
- Static single assignment (SSA) form
- Infinite registers

Syntax

- @ : global id, % : local id
 - `call i32 @puts(ptr %x)`
 - `%x = %a + 3`
- More details in Appendix



Preface

ENVIRONMENT SETUP

Installing Docker (GUI)

Docker Desktop

- Easiest way to install Docker on GUI-supported OS
 - Windows (+WSL)
 - Mac
 - Linux with GUI
- Download
 - <https://www.docker.com/products/docker-desktop/>

Installing Docker (CLI)

Official manual

- <https://docs.docker.com/engine/install/>
- Select your OS in the list to see the instructions

Install Docker Engine

This section describes how to install Docker Engine on Linux, also known as Docker CE. Docker Engine is also available for Windows, macOS, and Linux, through Docker Desktop. For instructions on how to install Docker Desktop, see: [Overview of Docker Desktop](#).

Supported platforms

Platform	x86_64 / amd64	arm64 / aarch64	arm (32-bit)	ppc64le	s390x
CentOS	✓	✓		✓	
Debian	✓	✓	✓	✓	
Fedora	✓	✓		✓	
Raspberry Pi OS (32-bit)			✓		
RHEL	✓	✓			✓
SLES					✓
Ubuntu	✓	✓	✓	✓	✓
Binaries	✓	✓	✓		

Installing LLVM - Quick

Option 1. Quick start

- Execute ``.container/deploy.sh`` after setting up docker
- Script will automatically build the Dockerfile and launch a container (container_name: `advcmp`, user: `ubuntu`, P/W: `ubuntu`)
- Can customize `container\_name`
- One can open shell via
``docker exec -it <container-name> bash``

Installing LLVM - Manual

Option 2. Manual download

- Follow one of these instructions
 - Github release: <https://releases.llvm.org/>
 - APT: <https://apt.llvm.org/> (recommended)
 - RPM: <https://copr.fedorainfracloud.org/coprs/g/fedora-llvm-team/llvm-snapshots/>

- Required packages (N: llvm version)
 - libllvm-N, llvm-N, llvm-N-dev

- Recommended packages
 - lld-N, lldb-N, clang-N

- It is **strongly recommended** to use Ubuntu **container** or **VM**
 - Separate the installed LLVM from the default compiler of the host OS

Installing LLVM - Build

If you are using other OS

- Use Ubuntu container (Highly recommended!)

- Build from source
 - C/C++ compiler for bootstrapping required
 - Recommend to use `ninja-build`, `lld`, `ccache`
 - Manual installation can **OVERWRITE** existing binaries and libraries
 - Compilation process is **time** and **resource** consuming

- Please refer to the official documents and Appendix of this slide
 - <https://llvm.org/docs/CMake.html>



Project 1

PRINTING CFG

Printing CFG

Steps

1. Generate (emit) LLVM IR from the source
2. Use `dot-cfg` pass in `opt`
3. Convert generated dot file into an image (.png)

Emit LLVM IR

➤ `clang -S -emit-llvm -O0 -Xclang -disable-O0-optnone \`
`-fno-discard-value-names <source> -o <out.ll>`

➤ Options

- `-disable-O0-optnone`
 - › `opt` may not run correctly if `optnone` flag is set
- `-fno-discard-value-names`
 - › Needed if you want to see the variable names in the source code

Printing CFG (Cont.)

Run ``dot-cfg`` pass

- ``opt -passes=dot-cfg <input.ll> -disable-output``
- DOT files will be generated
 - Their name starts with ``.``(dot) and invisible in default
 - So, please use ``ls -a`` to check they are correctly generated

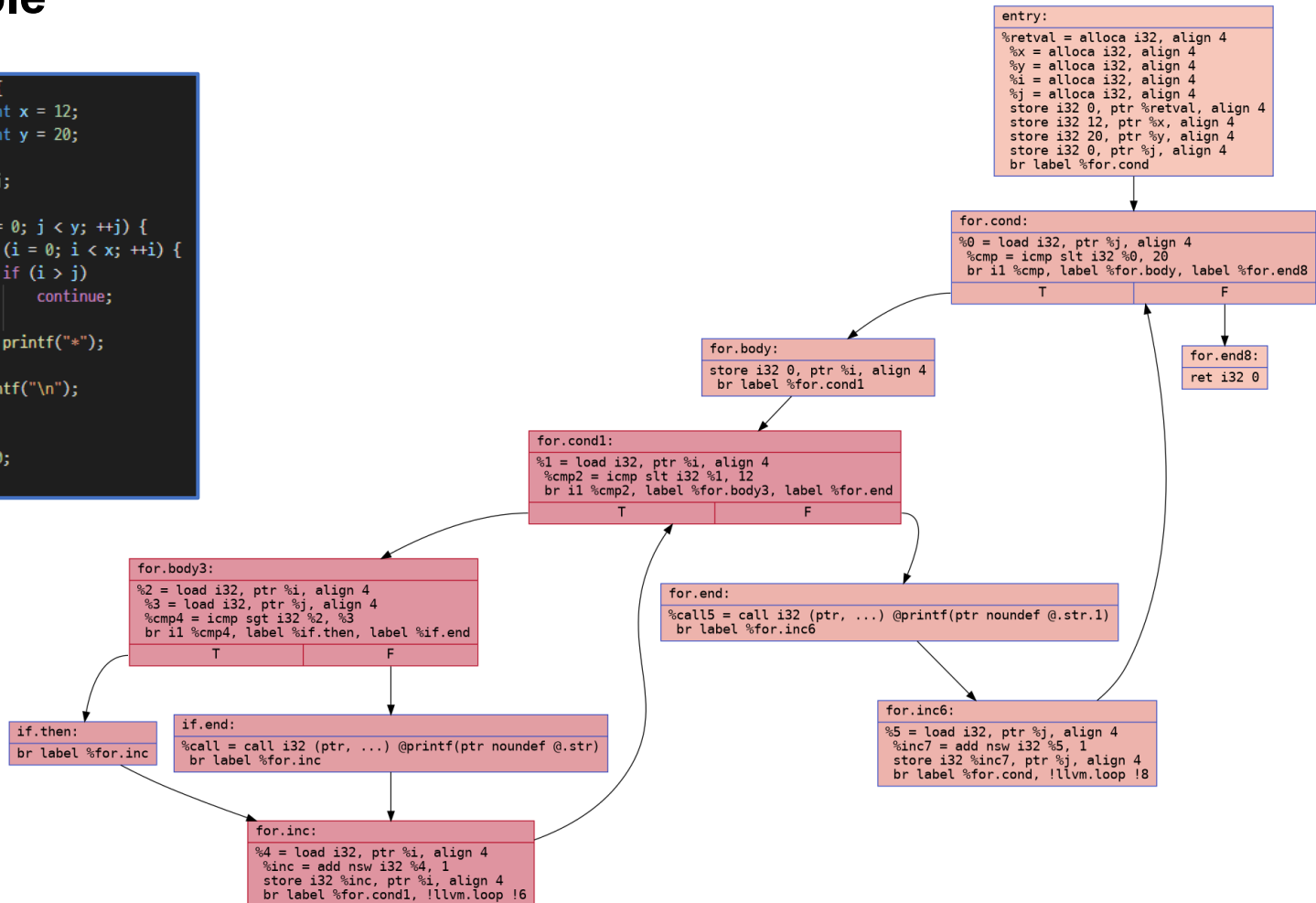
DOT to png

- One can convert DOT file into image
 - ``sudo apt install graphviz``
 - ``dot -Tpng <function_dot> -o <function.png>``

Project 1 : Printing CFG

Example

```
int main() {  
    const int x = 12;  
    const int y = 20;  
  
    int i, j;  
  
    for (j = 0; j < y; ++j) {  
        for (i = 0; i < x; ++i) {  
            if (i > j)  
                continue;  
            printf("*");  
        }  
        printf("\n");  
    }  
  
    return 0;  
}
```



CFG for 'main' function



Appendix. A

BUILD LLVM FROM SOURCE

Build from source

Download the source (based on llvm-17)

```
> git clone https://github.com/llvm/llvm-project.git
> git checkout -t release/17.x
```

Configure cmake

```
> mkdir llvm-project/build && cd llvm-project/build
> cmake -G Ninja ../llvm \
    -DCMAKE_CXX_STANDARD=17 \
    -DCMAKE_C_COMPILER=clang \
    -DCMAKE_CXX_COMPILER=clang++ \
    -DCMAKE_BUILD_TYPE=Release \
    -DCMAKE_INSTALL_PREFIX="/tmp/llvm" \
    -DLLVM_ENABLE_RTTI=ON \
    -DLLVM_TARGETS_TO_BUILD="host" \
    -DLLVM_ENABLE_ASSERTIONS=ON \
    -DLLVM_BUILD_LLVM_DYLIB=ON \
    -DLLVM_DYLIB_COMPONENTS="all" \
    -DLLVM_USE_LINKER="lld" \
    -DLLVM_CCACHE_BUILD=ON
```

Build from source

CMake options (<https://llvm.org/docs/CMake.html>)

- -DCMAKE_BUILD_TYPE=Release
 - Release / Debug / RelWithDebInfo / MinSizeRel
 - Debug build requires large memory space

- -DCMAKE_INSTALL_PREFIX="/tmp/llvm"
 - Configure installation path

- -DLLVM_ENABLE_RTTI=ON
 - Enable C++ run time type information (RTTI)
 - LLVM has its own more efficient RTTI system (<https://llvm.org/docs/HowToSetUpLLVMStyleRTTI.html>)
 - Turning off this option may cause dynamic casting failures

Build from source

CMake options (<https://llvm.org/docs/CMake.html>)

- `-DLLVM_TARGETS_TO_BUILD="host"`
 - Configure backend target language
 - Supports various architectures (AArch64, Mips, PowerPC, RISC-V, WebAssembly, ...)

- `-DLLVM_BUILD_LLVM_DYLIB=ON`
 - Build as a single dynamic library file (*.so)
 - If static, need to designate included libraries while building

- `-DLLVM_DYLIB_COMPONENTS="all"`
 - Same

- `-DLLVM_USE_LINKER="lld"`
 - Use LLVM's light and efficient linker (lld)

Build from source

CMake options (Tips)

➤ `-DLLVM_PARALLEL_LINK_JOBS=2`

- Specify parallelism of linker
- May need to restrict to avoid out of memory (OOM) especially debug build

➤ `-DLLVM_RAM_PER_LINK_JOB=512`

- Same (size in MB)

➤ `-DLLVM_ENABLE_PROJECTS="clang;mlir"`

- Can enable some LLVM based projects

Build from source

Build

```
> ninja build  
> ninja install
```

➤ LLVM will be installed at `/tmp/llvm`

Configure library

```
> echo "/tmp/llvm/lib" | sudo tee /etc/ld.so.conf.d/llvm.conf  
> sudo ldconfig
```



Appendix. B

LLVM DETAILS

LLVM IR

Syntax - prefix

- @ : global id, % : local id
 - `call i32 @puts(ptr %x)`
 - `%x = %a + 3`

Key instructions

- <https://llvm.org/docs/LangRef.html>
- Binary instructions
 - `add`, `sub`, `fsub`, `mul`, `fmul`, `udiv`, `sdiv`, `fdiv`, `shl`, `and`, `or`, ...
- Conversion instructions
 - `trunc`, `zext`, `sext`, ...
- Terminator instructions
 - All basic blocks should end with a terminator instruction.
 - `ret`, `br`, `callbr`, `switch`, `resume`, ...

LLVM IR

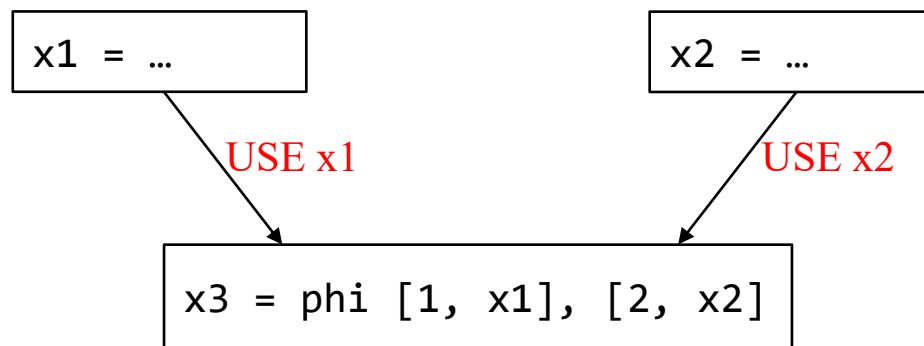
Key instructions

➤ phi (ϕ)

- Virtual function (node), which magically selects a value from the dynamic control flow
- Must precede any other non-phi instructions

```
Loop:      ; Infinite loop that counts from 0 on up...
%indvar = phi i32 [ 0, %LoopHeader ], [ %nextindvar, %Loop ]
%nextindvar = add i32 %indvar, 1
br label %Loop
```

- `Use` of each argument is happened on the edge between predecessor and current block



LLVM IR

Key instructions

➤ `alloca`

- Allocate stack memory
- **c.f.**, Heap memory : allocated through library call

➤ `load`, `store`

➤ `getelementptr` (GEP)

- Calculate the element's address from the base address, offset, and offset size
- Do NOT dereference (load)

```
int arr[8];  
int arrptr = arr;  
  
x1 = arr[3];  
x2 = arrptr[3];
```

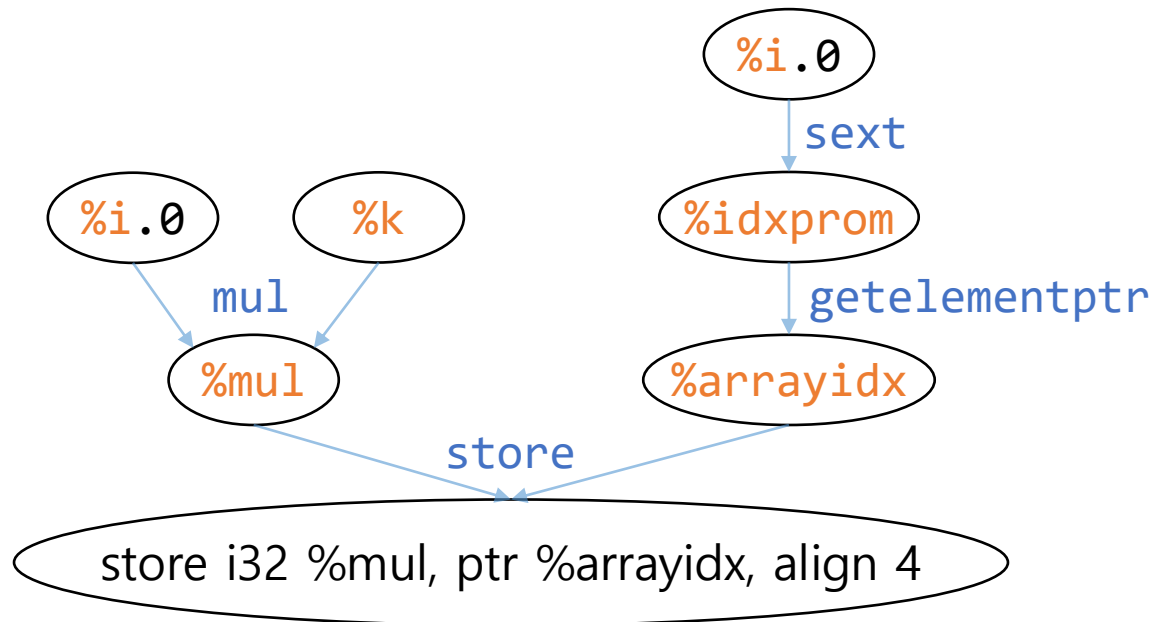
```
%1 = getelementptr [8 x i32], [8 x i32]* %addr, i64 0, i64 3  
%2 = getelementptr i32, i32* %addr, i64 3
```

LLVM IR

Structure

- `Instruction`s (edges) connecting `Value`s (nodes)

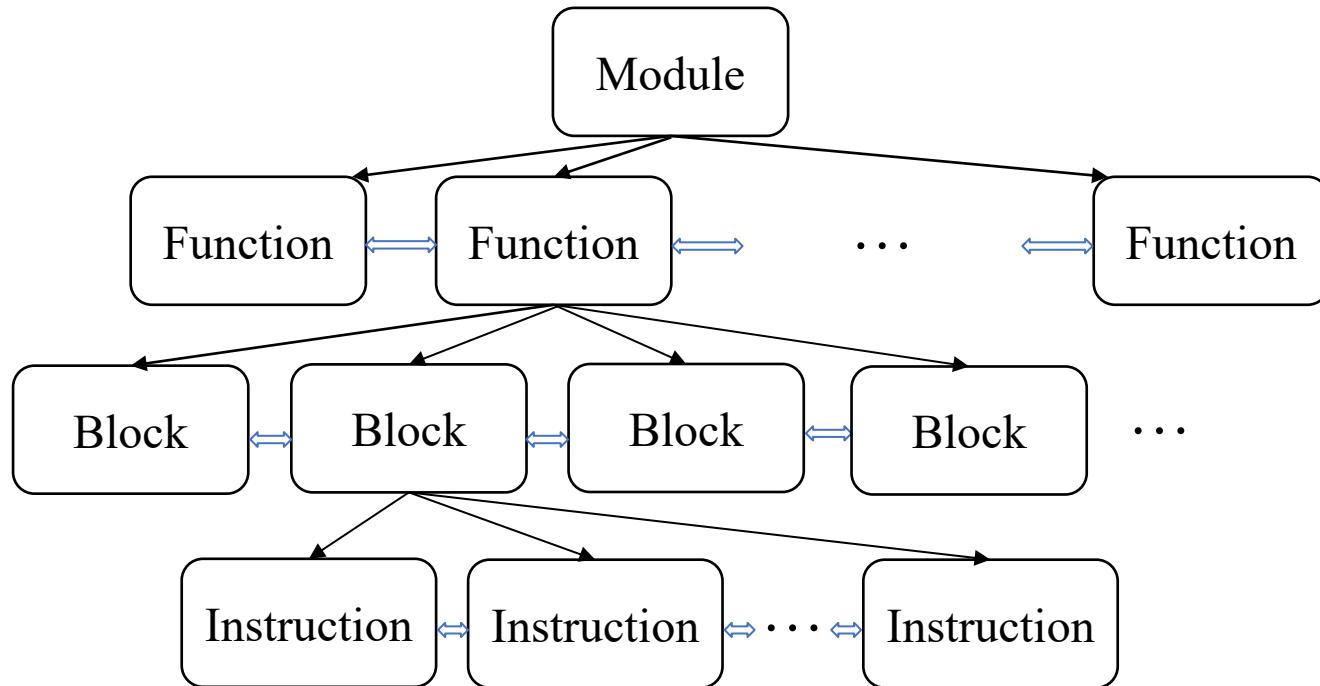
```
%mul = mul nsw i32 %i.0, %k  
%idxprom = sext i32 %i.0 to i64  
%arrayidx = getelementptr inbounds i32, ptr %a, i64 %idxprom  
store i32 %mul, ptr %arrayidx, align 4
```



LLVM IR

Structure

- `Instruction`s (edges) connecting `Value`s (nodes)
- `Value`s reside in doubly linked list (except `Module`)



Back-end

llc (CodeGen)

- <https://llvm.org/docs/CodeGenerator.html>
- Codes at ``llvm/lib/CodeGen``
 - Register allocation, SSA destruction (PHIElimination), ...
- Compiled as ``llc`` (LLVM compiler)
- Also uses ``SelectionDAG``, Machine-IR (MIR)
 - Please refer to *ch.8 Code Generation* of the Dragon book

Custom H/W (back-end)

- <https://llvm.org/docs/WritingAnLLVMBackend.html>
- One must specify the hardware specs through LLVM API

Middle-end

opt

- IR-to-IR optimization through passes
- LLVM had changed pass manager (PM)
 - We will use new PM throughout the projects
- Codes at [`llvm/lib/Transforms`](https://llvm.org/docs/Transforms), [`llvm/lib/Analysis`](https://llvm.org/docs/Analysis)

Custom pass

- <https://llvm.org/docs/WritingAnLLVMNewPMPass.html>
- We will handle this in our projects!

Security / Utils

- Code obfuscation (LINE, ORK)
- Fuzzing (Google, AFL)

Front-end

Clang

- LLVM's official C/C++ compiler
- <https://clang.llvm.org/docs/index.html>
- Codes at ``llvm-project/clang``

MLIR (Lattner, Google)

- Multi-Level IR
- For high-level IRs supporting domain specific languages
- Side project of LLVM
 - <https://mlir.llvm.org/>
- Codes at ``llvm-project/mlir``
 - Similar structure with LLVM
 - Also supports [python bindings](#)